



Managed by IHRD, a Govt. of Kerala undertaking

COLLEGE OF ENGINEERING
POONJAR

GROUP 1

PROJECT

SYNAPSE

PREPARED BY

Ihsana Ilyas

Albert Jossy

Abhishek TP

Adinan Azeez KT



DATABASE DESIGN DOCUMENT (DDD)

CONTENTS

▪ Introduction.....	2
-1.1 Purpose.....	2
-1.2 Scope.....	2
▪ Database Overview.....	2
-2.1 Database Description.....	2
-2.2 Database Architecture.....	2
▪ Database Design.....	3
-3.1 Entity Identification.....	3
-3.2 Database Tables.....	3
-3.3 Table Relationships.....	5
-3.4 ER Diagram.....	6
▪ Data Dictionary.....	7
-4.1 Train Table.....	7
-4.2 StationTable.....	7
-4.3 Delay History Table.....	7
-4.4 Weather History Table.....	8
-4.5 Prediction Log Table.....	8
▪ Database Operations.....	9
-5.1 Data Insertion.....	9
-5.2 Data Retrieval.....	9
-5.3 Data Update.....	9
▪ Conclusion.....	9
▪ References.....	10

1.INTRODUCTION

1.1 Purpose

The Database Design Document (DDD) describes the structure and organization of the database used in the Synapse train delay prediction system. It explains how data is stored, organized, and managed to support the various functionalities of the application. The database plays an important role in storing historical railway information, weather records, and prediction results that are required for training and executing the machine learning model.

This document serves as a technical reference for developers during database implementation, testing, and future maintenance. It also ensures that all project members have a common understanding of the database structure and the relationships between different data entities.

1.2 Scope

The database is designed to support all data-related activities of the Synapse system. It stores historical train information, weather data, station details, prediction logs, and metadata related to the machine learning model. These datasets are used for training the prediction model and generating accurate train delay predictions.

The database design focuses on data consistency, efficient retrieval, and future scalability. As additional trains and historical records become available, the database can be expanded without significant structural modifications.

2.DATABASE OVERVIEW

2.1 Database Description

The Synapse database serves as the central repository of the entire system. It stores all the information required by both the backend application and the machine learning model. Instead of repeatedly collecting the same information during every prediction request, the system retrieves historical records directly from the database, improving efficiency and reducing processing time.

A relational database model has been selected because it organizes data into structured tables with clearly defined relationships. This approach minimizes data redundancy, improves data consistency, and simplifies maintenance.

PostgreSQL has been proposed as the Database Management System (DBMS) due to its high reliability, scalability, security, and strong support for relational database operations. It also integrates well with Python-based backend frameworks such as FastAPI.

2.2 Database Architecture

The database is accessed only through the backend application. Users never communicate directly with the database. Instead, the frontend sends user requests to the backend through REST APIs. The backend retrieves the necessary historical train data and weather information, processes it using the Business Logic Layer, and forwards the prepared data to the machine learning model.

After prediction, the generated results are returned to the frontend and optionally stored in the Prediction Log table for future analysis.

3.DATABASE DESIGN

3.1 Entity Identification

The database has been designed using multiple entities, each responsible for storing a specific category of information. Dividing the database into separate tables improves maintainability and minimizes duplication of data.

Each entity represents an important component of the prediction process and is connected through primary and foreign key relationships.

3.2 Database Tables

Train

This table stores the basic details of trains considered by the system.

Attribute	Data Type	Description
Train_ID	INT	Primary Key
Train_Number	VARCHAR(10)	Train Number
Train_Name	VARCHAR(100)	Train Name

Station

Stores details of stations along the selected railway routes.

Attribute	Data Type	Description
Station_ID	INT	Primary Key
Station_Name	VARCHAR(100)	Station Name
Station_Code	VARCHAR(10)	Official Station Code

Delay_History

Stores historical delay records collected for machine learning training.

Attribute	Data Type	Description
Delay_ID	INT	Primary Key
Train_ID	INT	Foreign Key
Station_ID	INT	Foreign Key
Journey_Date	DATE	Date of Journey
Scheduled_Arrival	TIME	Scheduled Arrival
Actual_Arrival	TIME	Actual Arrival
Delay_Minutes	INT	Delay Duration

Weather_History

Stores weather information corresponding to train journeys.

Attribute	Data Type	Description
Weather_ID	INT	Primary Key
Station_ID	INT	Foreign Key
Date	DATE	Observation Date
Weather_Condition	VARCHAR(50)	Weather Description
Rainfall	FLOAT	Rainfall Amount
Temperature	FLOAT	Temperature

Prediction_Log

Stores prediction results generated by the machine learning model.

Attribute	Data Type	Description
Prediction_ID	INT	Primary Key
Train_ID	INT	Foreign Key
Prediction_Date	DATE	Prediction Date
Predicted_Delay	INT	Predicted Delay
Confidence	FLOAT	Confidence Score

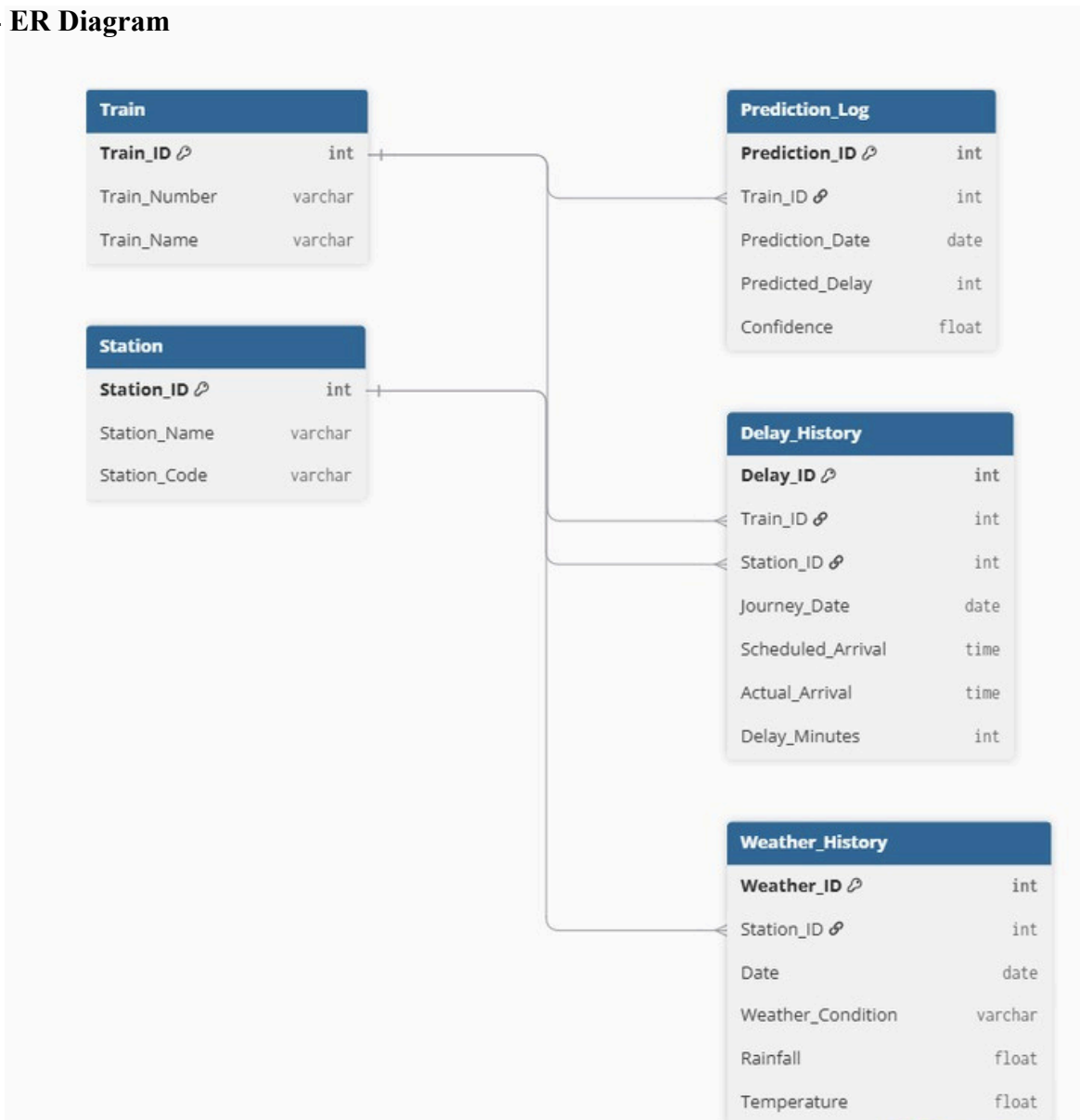
3.3 Table Relationships

The Train table is linked with both the Delay History and Prediction Log tables using Train_ID. Similarly, the Station table is connected with the Delay History and Weather History tables through Station_ID.

These relationships ensure that each delay record and weather record corresponds to a valid train and station. This design maintains data integrity and prevents duplicate or inconsistent records.

Parent Entity	Child Entity	Relationship
Train	Delay_History	One Train → Many Delay Records (1:M)
Station	Delay_History	One Station → Many Delay Records (1:M)
Station	Weather_History	One Station → Many Weather Records (1:M)
Train	Prediction_Log	One Train → Many Predictions (1:M)

3.4 ER Diagram



4.DATA DICTIONARY

4.1 Train Table

Field	Type	Description
Train_ID	INT	Unique identifier for each train
Train_Number	VARCHAR(10)	Official train number
Train_Name	VARCHAR(100)	Name of the train

4.2 Station Table

Field	Type	Description
Station_ID	INT	Unique identifier
Station_Name	VARCHAR(100)	Railway station name
Station_Code	VARCHAR(10)	Official station code

4.3 Delay history table

Field	Type	Description
Delay_ID	INT	Unique delay record
Train_ID	INT	References Train table
Station_ID	INT	References Station table
Journey_Date	DATE	Date of operation
Delay_Minutes	INT	Delay recorded in minutes

4.4 Weather History Table

Field	Type	Description
Weather_ID	INT	Unique weather record
Station_ID	INT	References Station table
Weather_Condition	VARCHAR(50)	Weather condition
Rainfall	FLOAT	Rainfall measurement
Temperature	FLOAT	Temperature value

4.5 Prediction Log Table

Field	Type	Description
Prediction_ID	INT	Prediction identifier
Train_ID	INT	References Train table
Prediction_Date	DATE	Date of prediction
Predicted_Delay	INT	Predicted delay
Confidence	FLOAT	Prediction confidence percentage

5.DATABASE OPERATIONS

The database performs several operations to support both the application and the machine learning model. These operations ensure that historical data is properly maintained and prediction results are efficiently generated.

5.1 Data Insertion

Historical train records, weather information, and station details are periodically inserted into the database after data collection and preprocessing. Newly generated prediction results are also stored in the Prediction Log table for future analysis and evaluation.

5.2 Data Retrieval

Whenever a user requests a prediction, the backend retrieves the relevant train information, historical delay records, and weather data from the database. These datasets are supplied to the machine learning model after feature engineering.

5.3 Data Update

The database is updated whenever new historical records become available. Updating the dataset enables periodic retraining of the machine learning model and improves the prediction accuracy over time.

5.4 Data Storage Strategy

The database follows a relational storage approach in which different categories of information are maintained in separate tables. This reduces redundancy and simplifies database maintenance. Historical records are preserved to support future machine learning training and performance analysis.

6.CONCLUSION

The proposed database design provides a structured and efficient mechanism for managing the data required by the Synapse train delay prediction system. By organizing information into well-defined relational tables, the database ensures data consistency, minimizes redundancy, and supports efficient retrieval during prediction.

The design also supports future scalability by allowing additional trains, stations, and historical datasets to be incorporated with minimal structural changes. As the machine learning model continues to evolve, the database will remain a reliable foundation for storing training data and improving prediction accuracy through continuous learning.

7. REFERENCES

1. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
3. PostgreSQL Global Development Group. PostgreSQL Documentation. Available at: <https://www.postgresql.org/docs/>
4. FastAPI Documentation. Available at: <https://fastapi.tiangolo.com/>
5. Scikit-learn Developers. Scikit-learn Documentation. Available at: <https://scikit-learn.org/>
6. Project Idea Report – Synapse: AI-Powered Train Delay Prediction System using Machine Learning.